

COMP3320/6464 High-Performance Scientific Computing

Shared Memory Parallel Programming – OpenMP

Dr Haibo Zhang
School of Computing
Australian National University



Australian
National
University

ANU College of
Engineering & Computer Science

Revised Hurdles for passing COMP3320/6464

- remove requirement to achieve at least 27 out of 60 across the combined score of the mid-semester test and the final exam

Only two remaining hurdles to pass this course

- at least 45% on the final written exam.
- at least 50 out of 100 in your final mark.

Final exam

- Tuesday, 12/11/2024

This week

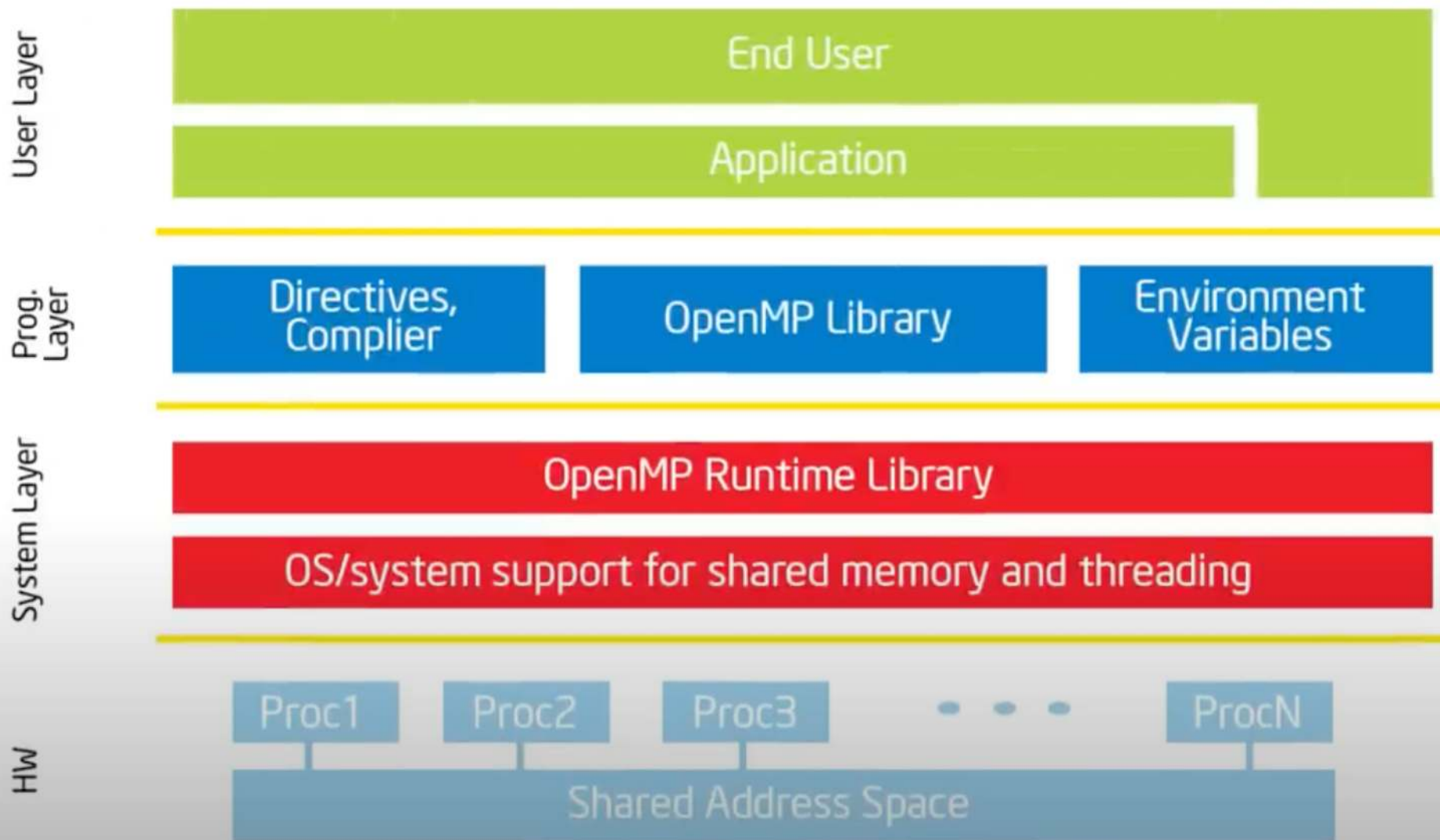
- Monday: OpenMP basics
- Tuesday: OpenMP advanced
- Wednesday & Thursday: lab 4 on OpenMP

- **Using OpenMP – Portable Shared Memory Parallel Programming**, B. Chapman, G. Jost, R. van der Pas, Chapter 1, 2, 3 & 4
- **Using OpenMP – The Next Step**, R. van der Pas, E. Stotzer, and C. Terboven, Chapter 1
- <http://www.openmp.org/>
- *Introduction to High Performance Computing for Scientists and Engineers*, Hager and Wellein, Chapter 6 & 7
- *High Performance Computing*, Dowd and Severance, Chapter 11
- *Parallel Programming in OpenMP*, R. Chandra, L.Dagum, D.Kohr, D.Maydan. J.McDonald, R.Menon

- Explicit thread programming (pthreads) is messy
 - low-level primitives
 - not easy to port
 - significant amount of boiler-plate code
 - used by system programmers, but ...
 - ... application programmers have better things to do!
- Many application codes can be supported by higher level constructs with the *same performance*
 - led to proprietary *directive* based approaches of Cray, SGI, Sun, etc.
- OpenMP is an API for shared memory parallel programming targeting Fortran, C and C++
 - standardizes the form of the proprietary directives
 - avoids the need for explicitly setting up mutexes, condition variables, data scope, and a good part of explicit initialization

- Specifications maintained by OpenMP Architecture Review Board (ARB)
 - members include AMD, Intel, Fujitsu, IBM, NVIDIA ... cOMPunity
- Versions 1.0 (Fortran '97, C '98), 1.1 and 2.0 (Fortran '00, C/C++ '02), 2.5 (unified Fortran and C, 2005), 3.0 (2008), 3.1 (2011), 4.0 (2013), 4.5 (2015), 5.0 (2018)
- Comprises compiler directives, library routines and environment variables
 - C directives (case sensitive)
`#pragma omp directive_name [clause-list]`
 - library calls begin with `omp_`
`void omp_set_num_threads(nthreads)`
 - environment variables begin with `OMP_`
`export OMP_NUM_THREADS=4`
- OpenMP requires compiler support
 - activated via `-fopenmp` (gcc) or `-qopenmp` (icc) compiler flags

OpenMP Basic Defs: Solution Stack



- OpenMP uses a fork/join model, i.e. programs execute serially until they encounter a `parallel` directive:
 - this creates a group of threads
 - the number of threads dependent on an environment variable or set via function call
 - the main thread becomes master with thread id 0

```
#pragma omp parallel [clause-list]
/* structured block */
```

- Each thread executes the *structured block*

A block of one or more statements with **one point entry at the top** and **one exit point at the bottom**.

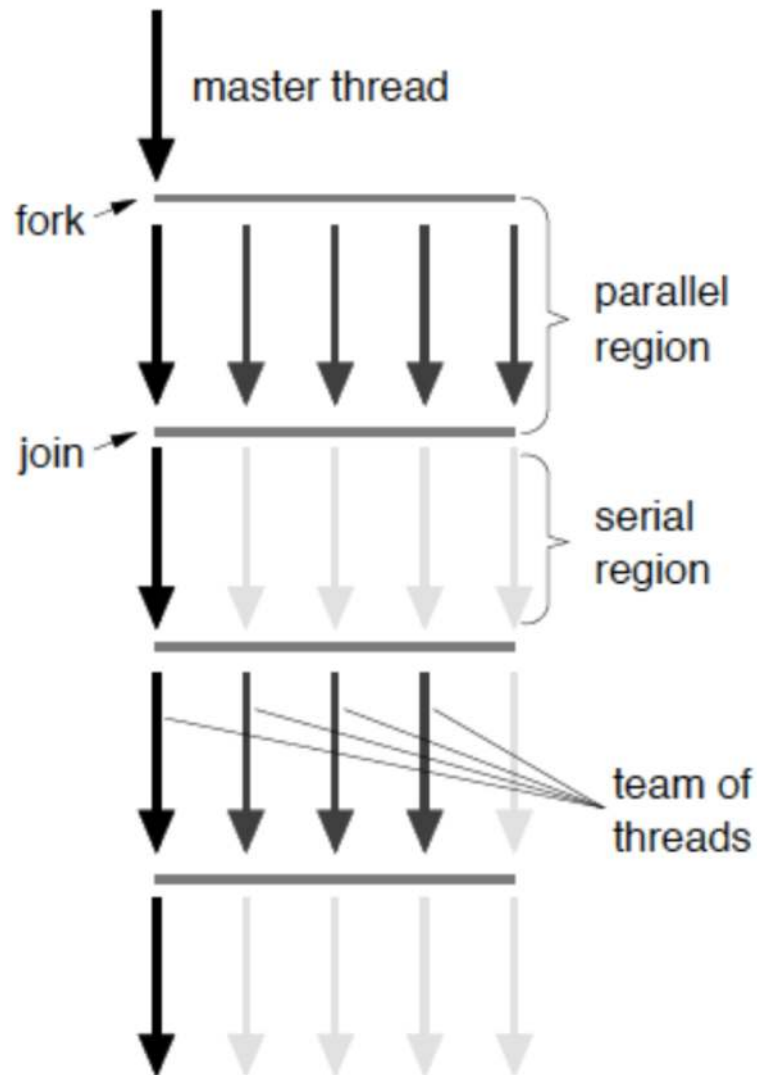
The Parallel Directive

- OpenMP uses a fork/join model, i.e. programs execute serially until they encounter a `parallel` directive:
 - this creates a group of threads
 - the number of threads dependent on an environment variable or set via function call
 - the main thread becomes master with thread id 0

```
#pragma omp parallel [clause-list]
/* structured block */
```

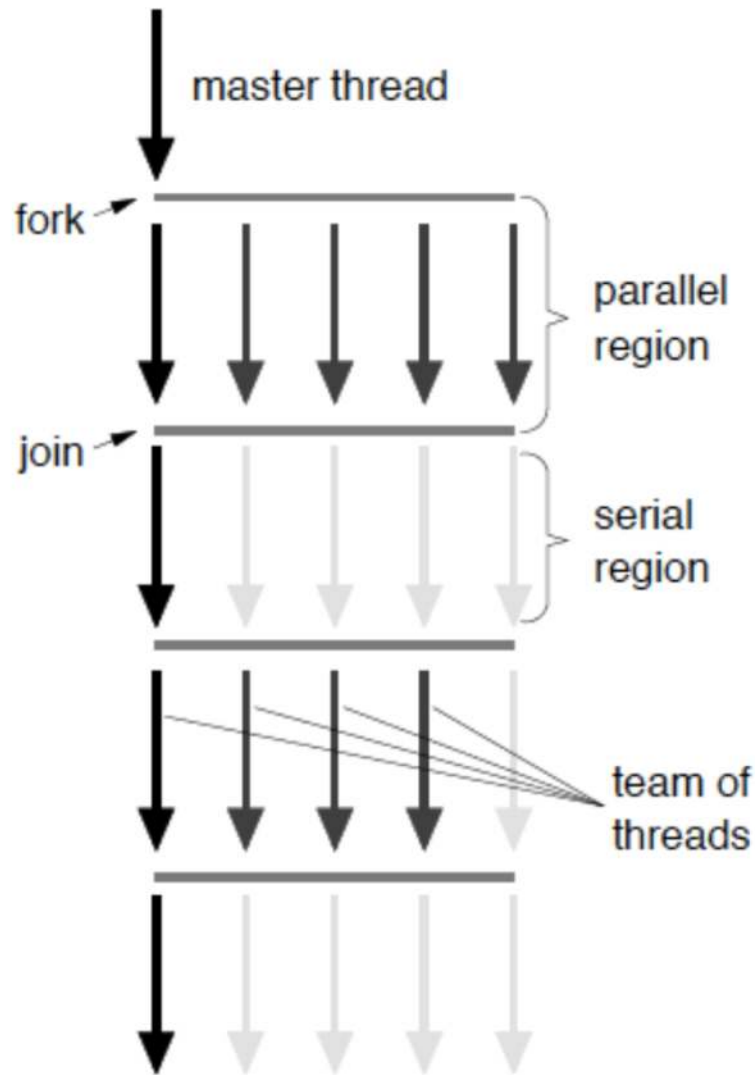
- Each thread executes the *structured block*
- In C/C++ this is a brace-enclosed (`{ code }`) sequence of statements and declarations.

The OpenMP Execution Model



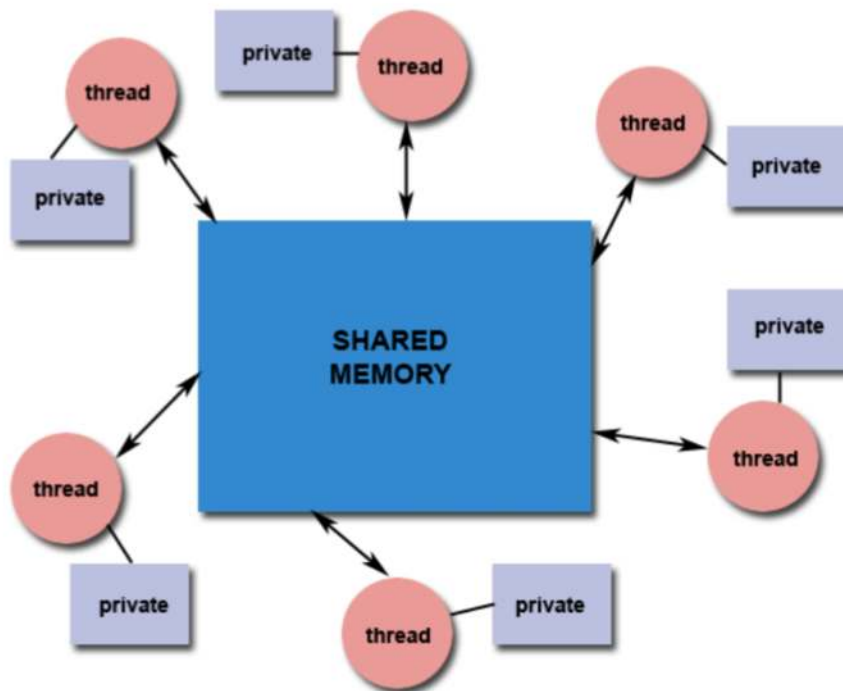
- An OpenMP starts in serial mode with one thread executing the serial code (*master thread*)
- At the beginning of the parallel region additional threads are created (*forking* from the master) by the runtime system forming a *thread team*
- All threads are active in the parallel regions, executing the program in parallel.
- At the end of the parallel region threads are *joined*, with only the master continuing through the serial portion.
- This is called the *fork-join* model.

The OpenMP Execution Model

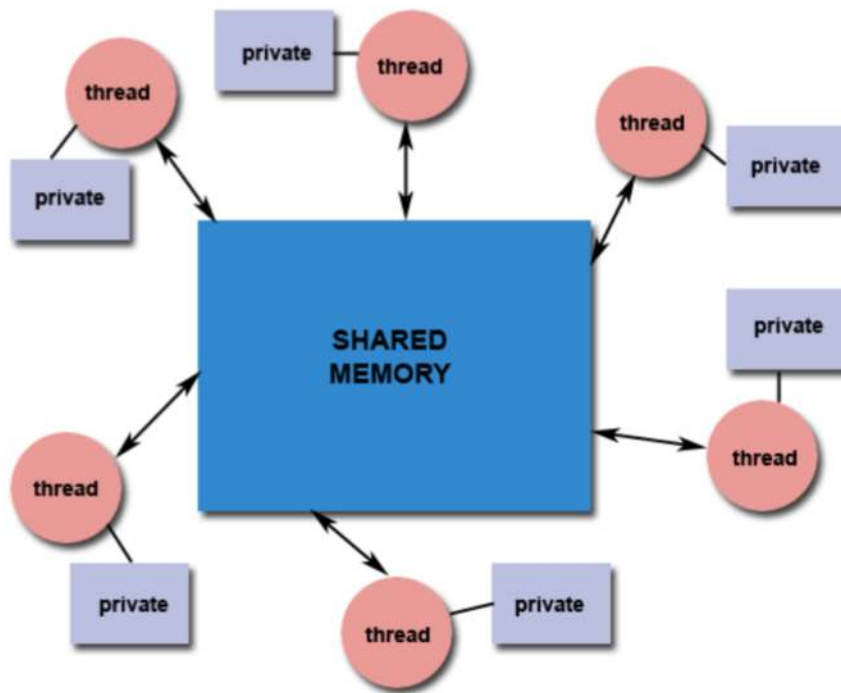


- The number of threads execution in the parallel region can be set through the `OMP_NUM_THREADS` environment variable.
- If the number of threads need to be more dynamic, the `omp_set_num_threads` may be used *prior* to a parallel region.
- An alternative is to use the `num_threads<nt>` clause on the `parallel` directive.
- Because of the join operation, the end of the parallel region is an *implicit synchronization point* (barrier).

The OpenMP Memory Model



- Some similarities with the pthreads memory model, but distinction between *private* and *shared* is much clearer.
- Whether a variable is private or shared as well as their initialization can be defined by *default rules*
- These can also be explicitly controlled through appropriate clauses on a construct.
- It is recommended not to rely on the default rules and explicitly label or “scope” variables.



Private and Shared Variables

- Private variables can be accessed only by the owning thread, no other thread may interfere.
- Threads may even use the same name for a private variable without the risk of any conflict.
- Each thread has read and write access to the same shared variable, that is only one instance of a given shared variable exists.
- Global or static variables are shared by default.

```
int g = 0; // g is shared
int main(){
    int i = 0; // i is shared
    static int a = 7; // a is shared
    #pragma omp parallel
    {
        int b = a + i + g; // b is
            private
        ...
    }
    return 0;
}
```

Default Rules

- Variables declared outside the parallel region are shared by default.
- Global and static variables are also shared by default.
- Variables declared inside the parallel region are private by default.

Data sharing clauses

```
int x = 5;
int y = 20;
int z[NUM_THREADS] = {0};
#pragma omp parallel default(none) shared(z)
    private(x) firstprivate(y)
{
    x = 10; // x is undefined on entry, but now set
           // to 10
    z[omp_get_thread_num()] = omp_get_thread_num();
    int w = x + y + z[omp_get_thread_num()]; // y pre-
           // initialized to a value of 20
    ...
    y = 30 // firstprivate var may be modified
}
```

- The private (shared) clause may be used to make a variable thread private (shared).
- One needs to be careful with initialization.
- The firstprivate clause guarantees that all threads have a pre-initialized copy of a variable
- The default clause is used to give a default data sharing attribute (none, shared, private) to all variables.
- When default(none) is used, the programmer is forced to specify data-sharing attributes for all variables in the construct.

Clauses are used to specify

- **Conditional Parallelization:** to determine if parallel construct results in creation of threads

`if(scalar expression)`

- **Degree of concurrency:** explicit specification of the number of threads created

`num_threads(integer_expression)`

- **Data handling:** to indicate if specific variables are local to thread (allocated on its stack, the default), global, or "special"

`private(variable_list)`

`shared(variable_list)`

`firstprivate(variable_list)`

Restrictions on the `parallel` directive

- A program that branches into or out of a parallel region is nonconforming, that is its behaviour is undefined.
- A program must not depend on any ordering of the evaluation (or side effects of evaluation) of the clauses of the `parallel` directive.
- At most one `if` clause can appear on the directive.
- At most one `num_threads` clause can appear on the directive.
- In C++, a `throw` inside a parallel region must cause execution to resume in the same region, and it must be caught by the same thread that threw the exception.
- in Fortran, unsynchronized use of I/O statements by multiple threads on the same unit has undefined behaviour.

The Work-Sharing Directives



Functionality	Syntax in C/C++	Syntax in Fortran
Distribute iterations over the threads	#pragma omp for	!\$omp do
Distribute independent work units	#pragma omp sections	!\$omp sections
Only one thread executes the code block	#pragma omp single	!\$omp single
Parallelize array-syntax		!\$omp workshare

- Used to distribute work among threads in a team.
- They specify the way the work has to be distributed among threads.
- Work-sharing directive must bind to a parallel region, otherwise is simply ignored.
- Work-sharing constructs do not have a barrier at entry.
- By default, a barrier is implemented at the end of the work-sharing region. The programmer can suppress the barrier with use of the `nowait` clause.

The for Work-Sharing Directive



- Used in conjunction with `parallel` directive to partition the `for` loop immediately afterwards

```
#pragma omp parallel shared(n)
{
    #pragma omp for
    for (i = 0; i < n; i++) {
        printf("Thread %d , iteration %d\n",
              omp_get_thread_num(), i);
    }
} /* End of parallel region */
```

- The loop index (`i`) is made private by default
- Only two directives plus the sequential code (code is easy to read/maintain)
- Limited to loops where number of iterations can be counted

The for Work-Sharing Directive



- Used in conjunction with `parallel` directive to partition the `for` loop immediately afterwards

```
#pragma omp parallel shared(n)
{
    #pragma omp for
    for (i = 0; i < n; i++) {
        printf("Thread %d , iteration %d\n",
              omp_get_thread_num(), i);
    }
} /* End of parallel region */
```

- The loop index (`i`) is made private by default
- Only two directives plus the sequential code (code is easy to read/maintain)
 - Limited to loops where number of iterations can be counted
- There is implicit synchronization at the end of the loop
 - Can add a `nowait` clause to prevent synchronization

The for Work-Sharing Directive



```
Thread 0 executes loop iteration 0
Thread 0 executes loop iteration 1
Thread 0 executes loop iteration 2
Thread 3 executes loop iteration 7
Thread 3 executes loop iteration 8
Thread 2 executes loop iteration 5
Thread 2 executes loop iteration 6
Thread 1 executes loop iteration 3
Thread 1 executes loop iteration 4
```

- The order in which threads execute is not predictable (OS scheduled).
- The way to map iterations to threads can be specified by the programmer (see later `schedule` clause).
- If the programmer does not specify the mapping between threads and iterations, the compiler decides which strategy to use.

The for Work-Sharing Directive



```
int x=0;
#pragma omp parallel num_threads(4)
{
    #pragma omp for lastprivate(x)
    for (int i = 0; i < iters; i++)
    {
        c[i] = a[i] * b[i];
        x=i;
        printf("Thread %d executes iteration %d\n", omp_get_thread_num(), i);
    }
}
```

The *lastprivate clause* is used with a parallel loop construct to ensure that the value of a variable is updated in the master thread with the value from the last iteration of the loop that was executed.

The sections Work-Sharing Directive

- Consider partitioning of fixed number of tasks across threads
- Each section must be a structured block that is independent from the other sections.
- Separate threads will run taskA and taskB
- Illegal to branch in or out of section blocks

```
#pragma omp parallel
{
    #pragma omp sections
    {
        #pragma omp section
        taskA()

        #pragma omp section
        taskB()

    } /* End of sections block */
} /* End of parallel region */
```

- Much less common than for loop partitioning
- Explicit programming naturally limits number of threads (scalability)
- Potential load imbalance

The single Work-Sharing Directive



- This directive specifies that only one thread must execute the code in structured block following it.
- It does not state which thread should execute the code.

```
#pragma omp parallel shared(a,b,n)
{
    #pragma omp single
    {
        a = 10;
    } /* A barrier is automatically inserted here */
    #pragma omp for
    for(i = 0; i < n; ++i)
    {
        b[i] = a;
    } /* Another barrier is automatically inserted here */
}
```


Combined parallel Work-Sharing Directives



Full version	Combined construct
<pre>#pragma omp parallel { #pragma omp for for-loop }</pre>	<pre>#pragma omp parallel for for-loop</pre>
<pre>#pragma omp parallel { #pragma omp sections { [#pragma omp section] structured block [#pragma omp section structured block] ... } }</pre>	<pre>#pragma omp parallel sections { [#pragma omp section] structured block [#pragma omp section structured block] ... }</pre>

- When there is only one work-sharing directive it can be combined with the `parallel` one to improve readability.
- Only clauses that are allowed by both the `parallel` and the specific work-share directive are allowed, otherwise the code is illegal.
- The compiler may optimize code further (e.g. remove redundant barriers).

- OpenMP fork/join model
- OpenMP memory model
- Parallel directive
 - **Degree of concurrency:** num_threads()
 - **Data handling:** private, shared, firstprivate, default
- for work-sharing directive
 - Distribute work among threads in a team
 - **No loop carried dependence**
- sections work-sharing directive
 - Partition work among threads
 - Potential load imbalance
- single work-sharing directive
 - Only one thread must execute it
 - Not restrict which thread should execute it.